mshqc: A C++20 Quantum Chemistry Engine for Orbital-Optimized Møller–Plesset Perturbation Theory

15 October 2026

Summary

`mshqc` is a high-performance quantum chemistry library that implements a focused set of wave-function-based electronic structure methods in a compact C++20 core with a low-overhead Python interface built on `nanobind` [@jakob2022nanobind]. The library provides restricted, unrestricted, and restricted open-shell Hartree–Fock references (`RHF`, `UHF`, `ROHF`) [@roothaan1951new; @pople1954self; @roothaan1960self] accelerated by Pulay's Direct Inversion in the Iterative Subspace (`DIIS`) [@pulay1980convergence; @pulay1982improved] and Superposition of Atomic Densities (`SAD`) initial guesses, alongside second- and third-order Møller–Plesset perturbation theory in both canonical (`RMP2`, `UMP2`, `RMP3`, `UMP3`) [@moller1934note; @pople1976theoretical; @head1988mp2] and orbital-optimized forms (`OMP2`, `OMP3`) [@neese2009assessment; @bozkaya2011quadratically; @bozkaya2011omp3].

To eliminate steep memory and computational bottlenecks in four-center electron repulsion integrals (ERIs) evaluated via `libcint` [@sun2015libcint], `mshqc` unifies three integral representations across all implemented SCF and correlated solvers: exact four-center tensors (`exact`) with Schwarz prescreening [@haser1989improvements], pivot-based Cholesky decomposition (`cholesky`) [@beebe1977simplifications; @koch2003reduced], and auxiliary-basis Density Fitting / Resolution of the Identity (`df`) [@feyereisen1993use; @weigend2002fully]. High-dimensional tensor contractions in the correlated kernels are dispatched to `TBLIS` [@matthews2017high], while orbital rotations in `OMP2` and `OMP3` are driven by a matrix-free Steihaug–Toint trust-region Second-Order SCF (`SOSCF`) optimizer.

Statement of need

Orbital-optimized Møller–Plesset perturbation methods (`OMP2` and `OMP3`) relax the reference orbitals in the presence of dynamical electron correlation, restoring Hellmann–Feynman satisfaction for one-electron properties, eliminating artificial spin contamination in open-shell radicals, and mitigating first-order orbital relaxation errors [@neese2009assessment; @bozkaya2011quadratically; @bozkaya2011omp3]. However, because each macro-iteration of an orbital-optimized calculation requires re-transforming two-electron integrals and re-evaluating $O(N^5)$ (`OMP2`) or $O(N^6)$ (`OMP3`) amplitude and response contractions, naive implementations quickly become memory- and compute-bound.

Researchers and students who wish to prototype new orbital-optimization algorithms, three-index tensor factorizations, or custom contraction schedules face a steep barrier to entry in large production suites. General-purpose ecosystems are designed for broad method coverage, which necessarily introduces deep abstraction layers between the high-level Python driver and the low-level numerical kernels. `mshqc` addresses this need by providing a compact, inspectable C++20 codebase where the SCF engines, three-index `cholesky`/`df` integral transformers, `TBLIS` tensor contractions, and trust-region `SOSCF` orbital optimizers are unified under a clean, two-tier API. Users can execute complete multi-method pipelines in a single Python call via `mshqc.run()`, or instantiate individual C++ classes directly to inspect and modify intermediate tensors.

State of the field

Mature open-source quantum chemistry packages such as `Psi4` [@smith2020psi4] and `PySCF` [@sun2020recent] offer comprehensive electronic structure capabilities, including canonical and orbital-optimized perturbation theory. In `Psi4`, orbital-optimized methods are implemented within the compiled `fnocc` and `occ` modules, whereas `PySCF` combines C-level integral evaluation with Python/NumPy tensor operations. While both ecosystems excel as general-purpose production tools, modifying their internal integral transformation pipelines or orbital Hessian preconditioners requires navigating hundreds of thousands of lines of code.

Conversely, educational or lightweight quantum chemistry projects typically implement only canonical `RHF` and `RMP2` in pure Python, lacking open-shell references (`UHF`/`ROHF`), third-order correlation (`MP3`/`OMP3`), point-group symmetry reduction, or rank-reduced three-index integrals (`cholesky` and `df`). `mshqc` bridges the gap between minimalist teaching codes and monolithic production suites: it maintains a small, single-purpose C++20 codebase while delivering execution speeds that match or exceed production packages on `OMP2` and `OMP3` workloads.

Software design

`mshqc` is organized into modular C++20 layers exposed to Python through `nanobind` [@jakob2022nanobind] with zero-copy `Eigen` [@guennebaud2010eigen] matrix interoperability and automatic Global Interpreter Lock (`GIL`) release during compute-intensive calls:

1. **Molecular Symmetry & Basis Infrastructure (`Molecule`, `BasisSet`, `PointGroup`, `PetiteList`):** Parses Gaussian basis set (`.gbs`) files bundled directly within the Python package—including the Dunning correlation-consistent families (`cc-pVXZ`) [@dunning1989gaussian] and matched `RI`/`JKFIT` auxiliary basis sets [@weigend2002fully]—with automatic contraction normalization [@helgaker2000molecular]. Automated point-group detection aligns molecular coordinates to canonical symmetry axes and constructs the `PetiteList` of symmetry-unique shell pairs [@atkins1970tables].
2. **Unified Integral Engine (`IntegralEngine`, `Screening`, `CholeskyERI`, `DFERI`):** Evaluates analytical one- and two-electron integrals in spherical harmonic or Cartesian representations via `libcint` [@sun2015libcint]. All solvers accept a unified `eri_method` configuration (`"exact"`, `"cholesky"`, or `"df"`), reducing two-electron storage from $O(N^4)$ four-center tensors to $O(N_{\text{aux}}N^2)$ three-center metric-decomposed matrices B_{pq}^{Q}.
3. **Self-Consistent Field (`RHF`, `UHF`, `ROHF`):** Shares a common base infrastructure (`BaseSCF`) supporting SAD initial guesses, Schwarz screening, symmetry-reduced Fock construction, and `DIIS` extrapolation. For open-shell systems under Density Fitting, exchange contractions exploit occupied-orbital rank projection to avoid $O(N_{\text{aux}}N^3)$ bottlenecks.
4. **Correlated & Orbital-Optimized Solvers (`RMP2`, `UMP2`, `OMP2`, `RMP3`, `UMP3`, `OMP3`):** Dispatches four-index tensor contractions to `TBLIS` [@matthews2017high] and three-index `df`/`cholesky` contractions to cache-blocked `OpenMP` + `BLAS` (`DGEMM`/`DSYRK`) kernels [@dagum1998openmp]. In `OMP2` and `OMP3`, non-redundant orbital rotation parameters κ_{ai} are optimized using a matrix-free Steihaug–Toint truncated conjugate gradient (`TrustRegionSOSCF`) solver coupled with unrelaxed and Z-vector response densities.

Validation and performance

All methods in `mshqc` have been validated against `Psi4` [@smith2020psi4] across closed-shell molecules and open-shell atoms. Table 1 summarizes single-point energies and wall-clock execution times on H_2O (C_{2v}) and open-shell O (^3P, multiplicity 3) using the `cc-pVTZ` basis set (`cc-pVTZ-RI` auxiliary basis for `df`, convergence threshold 10^{-9} Ha). Across exact, Cholesky, and density-fitted pipelines, `mshqc` reproduces reference energies to within $\leq 10^{-6}$ Ha for exact/Cholesky integrals and $\leq 1.5 \times 10^{-4}$ Ha for auxiliary-basis DF, while achieving 2.5× to 16.4× speedups over `Psi4`.

Table 1: Representative benchmark comparison between `mshqc` and `Psi4` using the `cc-pVTZ` basis set (`cc-pVTZ-RI` for `df`, threshold 10^{-9} Ha).

System	ERI Mode	Method	`mshqc` Energy (Ha)	`Psi4` Energy (Ha)	Time `mshqc` (s)	Time `Psi4` (s)	Speedup
H_2O ($^1A_1$)	`exact`	`RHF`	−76.057161957	−76.057161955	0.056	0.145	2.59×
H_2O ($^1A_1$)	`exact`	`OMP2`	−76.329269877	−76.329269880	0.688	1.740	2.53×
H_2O ($^1A_1$)	`exact`	`OMP3`	−76.338800547	−76.338800549	4.084	5.006	4.62×
H_2O ($^1A_1$)	`cholesky`	`OMP3`	−76.338800547	−76.338799277	7.787	5.524	7.02×
H_2O ($^1A_1$)	`df`	`RMP2`	−76.328931027	−76.328931022	0.005	0.082	16.40×
H_2O ($^1A_1$)	`df`	`OMP3`	−76.338632087	−76.338779922	2.347	5.612	16.17×
O (^3P, UHF)	`cholesky`	`OMP3`	−75.019697387	−75.019697383	8.333	1.509	4.53×
O (^3P, UHF)	`df`	`OMP3`	−75.019609447	−75.019686033	3.257	1.336	5.20×

Availability and documentation

`mshqc` is released under the Apache License 2.0 and hosted on GitHub (https://github.com/syahrulhidayat/mshqc), with precompiled Linux wheels built via automated GitHub Actions workflows. Documentation, quickstart tutorials, and troubleshooting guides are included in the `docs/` directory of the repository.

AI usage disclosure

During the preparation of this work, the author used generative AI tools (Gemini) to assist with code refactoring, performance profiling analysis, and manuscript editing. The author reviewed, validated, and tested all code and text against analytical quantum chemistry references and assumes full responsibility for the published work.

Acknowledgements

The author acknowledges the developers of `libcint` [@sun2015libcint] for analytical Gaussian integral evaluation and `TBLIS` [@matthews2017high] for transpose-free tensor contractions, as well as the `PySCF` [@sun2020recent] and

`Psi4` [@smith2020psi4] projects for providing reference values during numerical validation.

References